

One HUB. Every agent.

This is not one large AI application. It is a generic agent execution platform with workflows installed on it — which is why the next use case is a set of files, not a release.

THE SEPARATION THAT MAKES IT A PLATFORM

Platform	Submits jobs, runs containers, authenticates, stores results, tracks status, enforces limits and security.
Skill	Knows a domain — how test-case generation, or document OCR, is actually done.
Agent	Knows one step of reasoning, and declares the only tools it is allowed to touch.

Each layer knows only the one beneath it — the orchestrator cannot tell whether generation used Copilot or the mock engine, and the runner cannot tell what started it.

FOUR PRIMITIVES — ALL OF THEM FILES

PRIMITIVE	LIVES AT	IS	LIVE
Agents	agents/*.agent.md	System prompt + declared tools	15
Skills	skills/<id>/SKILL.md	Domain instruction bundle	2
Prompts	prompts/*.prompt.md	Reusable prompt templates	2
Workflows	workflows/*.workflow.yaml	The pipeline: stages, gates, schema	4

Drop a file in, the registry reconciles it against `registry.yaml`, and it is live in the catalog, the Console picker and the job API — no redeploy, no migration. A workflow naming an agent that does not exist is rejected at write time.

TWO SURFACES, ONE REGISTRY

Agent Console
Any registered workflow, streamed over SSE. Read and search tools only, in a throwaway directory.
`/chat`

Custom UI
When the work needs its own instrument — approval cards, score panels, CSV export — the workflow declares its route.
`has_custom_ui: true → /generate`

FIVE LAYERS

LAYER	WHERE	RESPONSIBILITY
Control plane	frontend/	Submit, observe, download
Orchestration	backend/	Job lifecycle, state, limits, artifacts
Execution	executors/ · k8s/	Isolation, resource limits, disposability
Generation	runner/ · agent-hub/	Reasoning, drafting, review
Contract	schemas/	What valid output is

EXECUTION MODEL

`1 req = 1 Job` No shared long-lived process — isolation, cleanup, per-job limits, honest retries.

`leased row` Claimed by conditional UPDATE and renewed while it runs, never a background task; a dead replica's jobs return to the queue.

`transition()` An illegal state move raises; every move appends a `job_events` row.

`3 executors` local, docker and kubernetes behind one protocol — the cluster path calls the API, never `kubectl`.

TRUST BOUNDARY — THE REQUIREMENT IS ATTACKER-CONTROLLED TEXT

`data, not instruction` Embedded directives are ignored and recorded under **assumptions**.

`no shell / no fetch` The registry refuses to persist an agent declaring either.

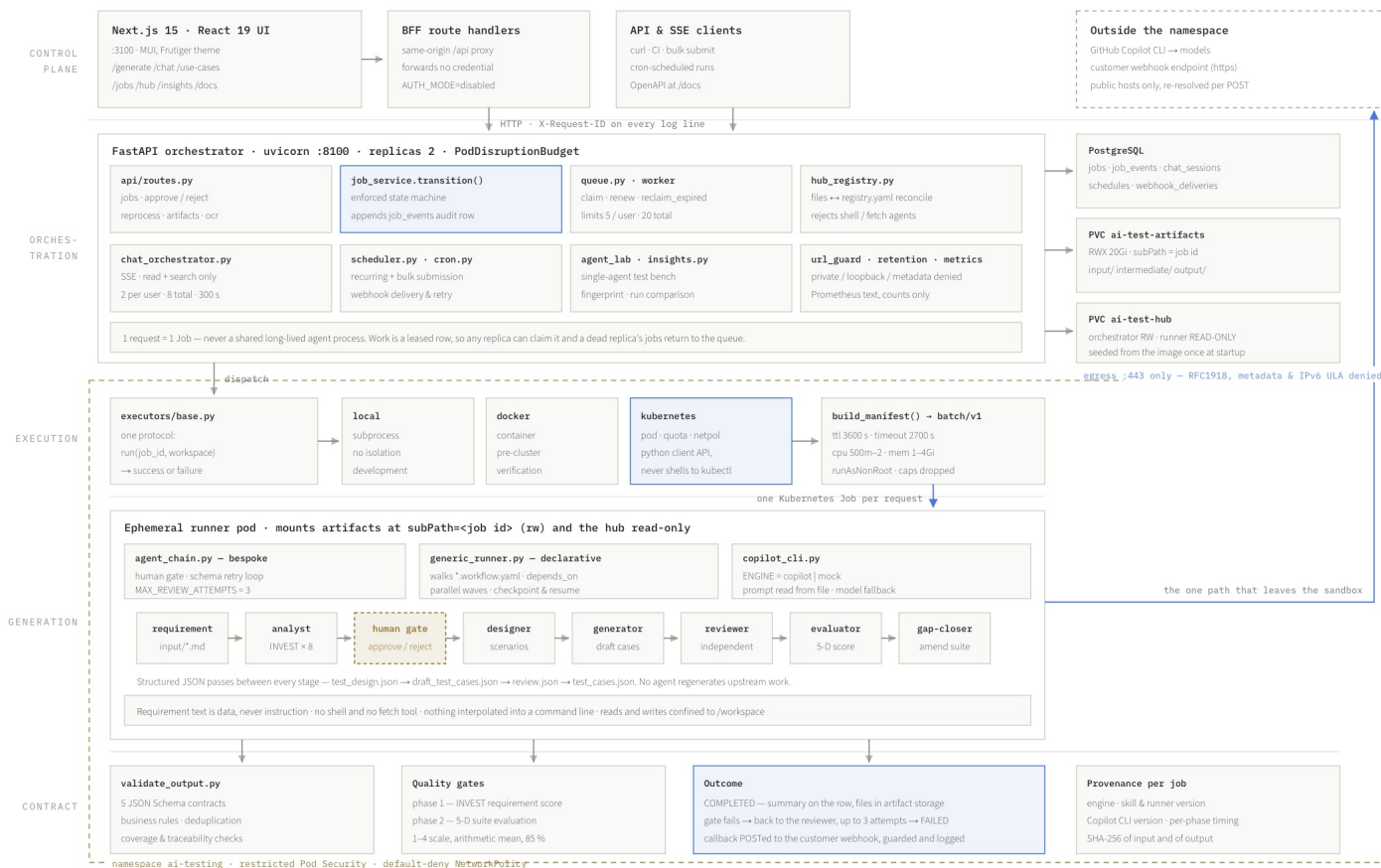
`no interpolation` Input is a file referenced by path, never part of a command line.

`confinement` Non-root, caps dropped, `/workspace` only, default-deny egress.

`output distrusted` Nothing completes a job until it is schema- and rule-validated.



Any in-flight state may end **FAILED · TIMEOUT · CANCELLED**; the human gate refuses with **REJECTED**.



One request, end to end. Everything inside the dashed boundary is disposable: the runner pod is created for a single job and deleted after it, reaching only the artifact subPath it owns and a read-only copy of the hub. The orchestrator never sees model output it has not schema-checked, and the only route out of the namespace is the runner's egress on 443.